Lab 5 – Shift Registers

1. **Wire up a shift register circuit.** Connect your 10-segment LED display, 220 Ω network resistor, SN74HC595 shift register, and ESP32, with shift register outputs Q_A through Q_H connected to 8 sequential LEDs on the display. Be sure to include the network resistor to limit current through the LEDs.

2. **Test the circuit with in-class code.** Using the `Shifter` class reviewed in lecture, write Python code that does the following:
   a. Instantiate a `Shifter` object that can be used to control outputs to the LED.
   b. Move a single active LED pixel in a *random walk* with a time step of 0.05 s. In a random walk, the pixel moves by either -1 or +1 position at each time step with equal probability. The active LED should be prevented from moving beyond the left or right edges of the display. The result should look something like a lightning bug trapped within the display.

3. **Create a Bug class.** Next, create a new class called `Bug` that will encapsulate the functionality from step 2. The `Bug` class must include the following attributes:

`timestep`	The time step size in seconds.
`isOn`	If False, the Bug should not be moving. If True, the Bug should be running.
`isWrapOn`	If False, the LED should be prevented from moving past either edge, as in problem 2. If True, LED "movement" should wrap around the display (when moving off one edge the lit LED should appear at the other edge).
`x`	The current active LED position in the display bar (from 0-7). The initial value should be 3.
`__shifter`	A `Shifter` object used to control the display. Note the double-underscore for this attribute, indicating that it is meant to be private.

   Your class must allow the first 3 attributes (`timestep`, `isOn`, and `isWrapOn`) to be passed as optional arguments when instantiating a new `Bug` object, with default values of 0.1, True, and False respectively.

   The `Bug` class must also implement the following methods:

`start()`	Start changing the LED position at the current speed
`stop()`	Stop changing the LED position (and turn off the LED)

   Do not modify the `Shifter` class for this step.

4. **Demonstrate your Bug class.** Write a program called `bug.py` that will instantiate a `Bug` object with default speed, position, and wrapping values, and do the following in an infinite loop:
   a. Monitor 3 GPIO inputs switches (let's call them s1, s2, and s3). Implement the "switches" using jumper wires connected to either ground (off) or 3.3V (on).
   b. When s1 is on, turn the Bug on (otherwise it must be off).
   c. When s2 changes state, flip the wrapping state (i.e. if the `Bug.wrap` was previously True, make if False, and *visa versa*).
   d. When s3 is on, increase the Bug speed by 5x (reduce the delay between steps by a factor of 5).

Instead of constantly monitoring the switches in your main loop, you are required to use IRQs with one or more callback functions that change the value of the appropriate object attribute. Debouncing your switch inputs is recommended, but not required for this lab.

Demonstrate your final system by taking a video, showing that you can turn the Bug on and off, flip the wrapping state, and change the movement speed.

To Submit:

Upload your final code and a short video demonstrating operation of the system in step 4.